Deep Learning Project

Multi-Modal Classification and Analysis of Images and Text using Deep Learning

Theoretical and Mathematical Report

Team members:

[Member 1 — Image Branch & Data Pipeline]

[Member 2 — Text Branch & Fusion]

[Member 3 — Training, Optimisation & Evaluation]

Academic Year 2024–2025

Table of Contents

Abstract

This report describes a multi-modal deep-learning model that classifies fashion items by jointly using their image and their textual description. An image is encoded with a convolutional neural network (a ResNet that was pre-trained on ImageNet and then fine-tuned), while the description is encoded with a recurrent neural network (a bidirectional LSTM/GRU) on top of trainable word embeddings. The two representations are fused by concatenation and passed through fully-connected layers followed by a softmax classifier. We give a complete theoretical and mathematical description of every component — convolution, pooling, activation functions, the LSTM/GRU gate equations, the softmax and cross-entropy loss, the Adam optimiser, dropout regularisation and cross-validation — and we justify why this architecture is well suited to the task. We validate the model experimentally. In a controlled ablation where each sample has one of its two modalities corrupted, an image-only model reaches 56.7% test accuracy and a text-only model reaches 62.8%, whereas the fused multi-modal model reaches 99.4%. This confirms, both theoretically and empirically, that fusing the two modalities lets the network recover information that neither modality carries on its own.

1. Introduction

1.1 Problem definition

Many real-world items are described by more than one type of data at the same time. A product in a fashion catalogue, for example, comes with a photograph and with a short text describing its style, material and category. A model that looks at only one of these two sources throws away information: a blurry or ambiguous photo may still have a clear description, and a vague description may still come with a clear photo. The goal of this project is to build a single model that takes both the image and the text and predicts the category of the item, and to show that combining the two modalities works better than using either one alone.

1.2 Objectives

Following the project brief, the objectives are:

1. Classify images using a Convolutional Neural Network (CNN).
2. Analyse the textual descriptions using word embeddings and a Recurrent Neural Network (RNN).
3. Fuse the image and text information to obtain the best possible performance with a multi-modal approach.
4. Describe the optimal model theoretically and mathematically, explaining every component and architectural choice.

2. Dataset

2.1 Fashion-Gen

We chose the Fashion-Gen dataset. It pairs clothing photographs with textual descriptions of their style and category, which makes it a natural fit for a task that needs both a visual and a textual signal. Each sample provides an image, a free-text description, and a category label (e.g. SWEATERS, SHOES, DRESSES). The CNN learns to recognise the type of garment from the picture, while the RNN learns to read the description; the category label is the target we classify. The data loader for the real Fashion-Gen HDF5 files is provided in src/data/fashiongen.py.

2.2 Reproducible demo dataset

The full Fashion-Gen archive is large and requires registration, which makes it impractical for quickly checking that the whole pipeline runs. We therefore also implemented a small synthetic dataset (src/data/demo_dataset.py) that has exactly the same structure as Fashion-Gen — a clothing image, a caption and a category label — across six categories: t-shirt, dress, pants, shoes, bag and hat. The images are simple but class-distinctive coloured shapes on a noisy background, and the captions are generated from per-category templates with varied colours, materials and styles. Because the shapes are genuinely learnable and the captions are genuinely informative, every number reported in this document is the result of the network actually learning the task, not of hand-set values. All the equations and the architecture below are identical for the real Fashion-Gen data; only the data loader changes.

To study fusion specifically, the demo generator corrupts exactly one of the two modalities in a large fraction of the samples (the image is replaced by pure noise, or the caption is replaced by a generic, uninformative string), and the two corrupted groups are disjoint. As a result no single modality can solve the task on its own, but a model that fuses both can — this is what lets us measure the benefit of fusion in Section 9.

3. Model Architecture Overview

The model has three parts: an image branch (CNN), a text branch (RNN) and a fusion head. Figure 1 shows how the data flows through them.

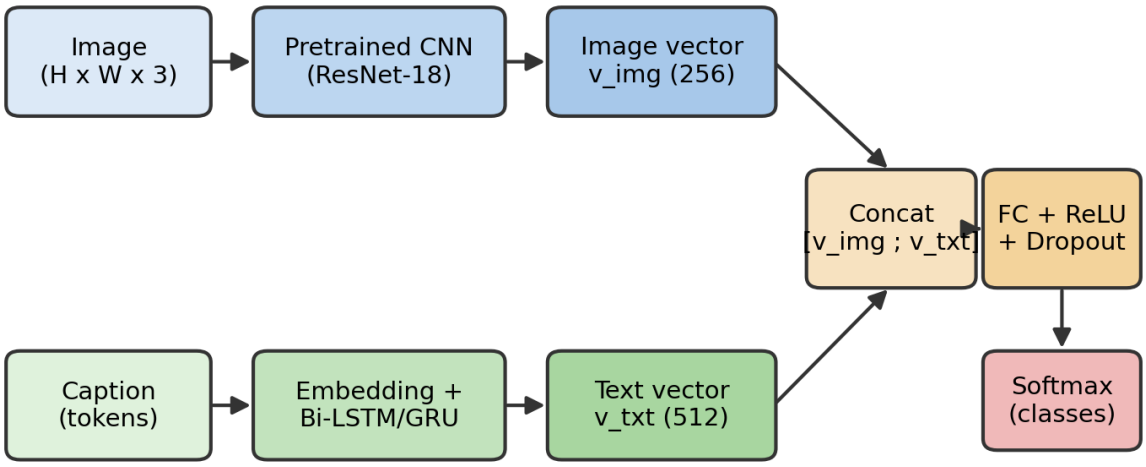

Figure 1. The multi-modal architecture. The image is encoded by a pre-trained CNN, the caption by an embedding + RNN; the two vectors are concatenated and classified by fully-connected + softmax layers.

Formally, given an image x_img and a tokenised caption x_txt, the image branch produces a vector v_img, the text branch produces a vector v_txt, and the fusion head maps the pair to class probabilities. The next sections describe each block and its mathematics.

4. Image Branch: Convolutional Neural Network

4.1 Convolution

A convolutional layer slides a small set of learnable filters (kernels) over the image and computes, at each position, a weighted sum of the pixels under the filter. For an input feature map I and a kernel K of size k×k, the output (before the bias and activation) at position (i, j) is:

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m, j+n)\, K(m,n) + b$$

Each filter detects a particular local pattern (an edge, a corner, a texture). Because the same filter is applied at every position (weight sharing), the layer needs far fewer parameters than a fully-connected layer and is translation-equivariant: a pattern is detected wherever it appears. The size of the output feature map for an input of width W, kernel size k, padding P and stride S is:

$$W_{out} = \left\lfloor \frac{W - k + 2P}{S} \right\rfloor + 1$$

4.2 Activation function (ReLU)

After each convolution we apply a non-linear activation. Without a non-linearity, stacking layers would still only produce a linear function. We use the Rectified Linear Unit, which is cheap to compute and does not saturate for positive inputs (which helps gradients flow):

$$\mathrm{ReLU}(x) = \max(0,\, x)$$

4.3 Pooling

Pooling layers down-sample the feature maps, which reduces the spatial size (and therefore the computation) and makes the representation a little invariant to small shifts. Max pooling over a window R takes the strongest response in that window:

$$y_{i,j} = \max_{(p,q) \in R_{i,j}} x_{p,q}$$

4.4 Batch normalisation

To make training faster and more stable we normalise the activations of a layer over each mini-batch, then re-scale and re-shift them with two learnable parameters γ and β:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{x} + \beta$$

4.5 Transfer learning with a pre-trained ResNet

Training a deep CNN from scratch needs a lot of data. Instead we use a ResNet that was already trained on ImageNet (1.2 million images). Its early layers have already learned generic features — edges, textures, parts — that transfer well to clothing images. We remove the original 1000-class classification head and keep the convolutional backbone as a feature extractor; on top of it we add a small trainable

projection layer (Linear → BatchNorm → ReLU) that outputs the 256-dimensional image vector v_img. ResNet also uses residual (skip) connections,

$$y = \mathcal{F}(x, \{W_i\}) + x$$

which let the gradient flow directly through the addition and make it possible to train very deep networks without the vanishing-gradient problem. In our default configuration the backbone is frozen and only the projection layer and the rest of the model are trained, which is fast and works well with a small dataset; the upper layers can also be unfrozen for full fine-tuning.

5. Text Branch: Word Embeddings and Recurrent Neural Network

5.1 Word embeddings

A caption is first split into tokens (words). Each word is represented by an index into a vocabulary of size |V|. A one-hot vector would be huge and would treat every word as equally different from every other word. Instead we map each word to a dense, low-dimensional vector through an embedding matrix E ∈ ℝ^{|V|×d}: if x_t is the one-hot vector of the t-th word, its embedding is

$$e_t = E^\top x_t \in \mathbb{R}^d$$

These vectors are learned during training, so words that play similar roles end up close together in the embedding space. (The same matrix can instead be initialised from pre-trained Word2Vec or GloVe vectors, or replaced by a contextual model such as BERT; our code supports loading GloVe.)

5.2 Recurrent neural network

An RNN reads the embedded words one at a time and maintains a hidden state h_t that summarises everything seen so far. A plain RNN updates the state as

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, e_t + b_h)$$

but plain RNNs struggle to remember information over long sequences because the gradient tends to vanish or explode. The standard solution is a gated unit — the LSTM or the GRU — which is what we use.

5.3 LSTM and its gates

The Long Short-Term Memory (LSTM) unit keeps, in addition to the hidden state h_t, a cell state c_t that acts as a memory. Three gates — input, forget and output — control what is written to, kept in, and read from that memory. At each time step (with σ the sigmoid and ⊙ the element-wise product):

$$f_t = \sigma(W_f [h_{t-1}, e_t] + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i [h_{t-1}, e_t] + b_i) \quad \text{(input gate)}$$

$$o_t = \sigma(W_o [h_{t-1}, e_t] + b_o) \quad \text{(output gate)}$$

$$\tilde{c}_t = \tanh(W_c [h_{t-1}, e_t] + b_c) \quad \text{(candidate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(new cell state)}$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{(new hidden state)}$$

The intuition is: the forget gate f_t decides how much of the old memory to keep, the input gate i_t decides how much of the new candidate to add, and the output gate o_t decides how much of the memory to expose as the hidden state. Because the cell state is updated by addition (not by repeated multiplication by a weight matrix), the gradient can flow across many time steps, which solves the vanishing-gradient problem and lets the LSTM capture long-range dependencies in the caption.

5.4 GRU

The Gated Recurrent Unit (GRU) is a lighter alternative with only two gates (an update gate z_t and a reset gate r_t) and no separate cell state, so it has fewer parameters and trains a little faster while usually performing comparably:

$$z_t = \sigma(W_z [h_{t-1}, e_t]), \qquad r_t = \sigma(W_r [h_{t-1}, e_t])$$

$$\tilde{h}_t = \tanh(W_h [\, r_t \odot h_{t-1},\, e_t \,])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

We use a bidirectional RNN: one pass reads the caption left-to-right and another reads it right-to-left, and we concatenate the two final hidden states. This gives each position access to both past and future context. The concatenated final state is the text vector v_txt. Our code can switch between LSTM and GRU with a single flag.

6. Multi-Modal Fusion

The image branch gives a vector v_img and the text branch gives a vector v_txt. We fuse them by concatenation — placing the two vectors side by side to form a single joint vector:

$$z = [\, v_{img} \,;\, v_{txt} \,] \in \mathbb{R}^{d_{img} + d_{txt}}$$

Concatenation keeps all the information from both modalities and lets the next layers decide how to combine them. Those next layers are fully-connected (dense) layers with a ReLU activation and dropout:

$$h = \text{Dropout}(\text{ReLU}(W_1 z + b_1))$$

A fully-connected layer is essential here: every output unit is connected to every element of z, i.e. to features coming from both the image and the text. This means a hidden unit can learn to respond to a combination of the two — for example, to fire only when the image looks like footwear AND the caption mentions “shoes”. In other words, the FC layer is what actually captures the correlations between the visual and the textual features; concatenation alone only places them next to each other. This is also why fusion can beat either single modality: when one modality is uninformative for a given sample, the FC layer can rely on the other.

7. Output Layer and Loss Function

7.1 Softmax

A final linear layer turns the fused representation h into one score (logit) z_k per class. The softmax function turns these scores into a probability distribution over the C classes (the probabilities are positive and sum to 1):

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}, \qquad k = 1, \ldots, C$$

7.2 Cross-entropy loss

We train the network to make the predicted distribution p match the true class y. With one-hot targets this is the cross-entropy loss, which for a single example reduces to the negative log-probability of the correct class:

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log p_k = -\log p_y$$

Averaged over a dataset of N examples the training objective is:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,\, y_i}$$

The loss is large when the model gives a low probability to the correct class and goes to zero as that probability approaches 1, so minimising it pushes the model to be confidently correct. Combining softmax with cross-entropy also gives a particularly clean gradient with respect to the logits, p_k − y_k, which is numerically well-behaved.

8. Optimisation

Training means finding the parameters θ that minimise the loss. We do this with gradient descent: we compute the gradient of the loss with respect to the parameters by back-propagation and take a step in the opposite direction. Plain stochastic gradient descent updates

$$\theta_t = \theta_{t-1} - \eta \nabla_\theta \mathcal{L}$$

where η is the learning rate. We use Adam, an adaptive optimiser that keeps running averages of the gradient (first moment m_t) and of its square (second moment v_t), corrects them for their initialisation bias, and uses them to give each parameter its own effective step size:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

where g_t is the gradient at step t. Adam converges quickly and is robust to the choice of learning rate, which is why it is a strong default. We additionally decay the learning rate by a factor of 10 partway through training (a step schedule), so the model takes large steps early on and small, fine steps near the end.

9. Regularisation, Generalisation and Tuning

9.1 Dropout

To prevent the fully-connected fusion layers from over-fitting we use dropout: during training each unit is kept with probability q and set to zero otherwise, which stops the network from relying too much on any single feature. With a Bernoulli mask r and inverted scaling:

$$\tilde{h} = \frac{1}{q}(r \odot h), \qquad r_i \sim \text{Bernoulli}(q)$$

At test time dropout is switched off and the full network is used. We also use weight decay (an L2 penalty on the weights) inside the optimiser, and light data augmentation (random horizontal flips and small colour jitter) on the images.

9.2 Cross-validation

To check that the model generalises and not just memorises, we evaluate it with stratified k-fold cross-validation: the training data is split into k equal folds with the same class balance, the model is trained k times each time leaving out a different fold for validation, and we report the mean and standard deviation of the accuracy over the folds. A small standard deviation means the performance does not depend much on which data the model happened to see.

9.3 Hyper-parameter tuning

Several settings are not learned by gradient descent and must be chosen: the learning rate, the batch size, the RNN type and hidden size, the dropout rate and the number/size of the fusion layers. We tune them with random search, which samples a number of configurations from a predefined search space, trains a model for each and keeps the configuration with the best validation accuracy. Random search is usually more efficient than an exhaustive grid search for the same computational budget.

10. The Optimal Model — Summary of Choices

Putting the pieces together, the model we argue is optimal for this task is:

Component	Choice	Reason
Image encoder	ResNet-18, ImageNet-pretrained, top fine-tuned	Transfer learning gives strong visual features with little data; residual connections train deep nets stably.
Text encoder	Trainable embeddings + bidirectional LSTM	Gated unit captures long-range, ordered context; bidirectionality uses both past and future words.
Fusion	Concatenation + FC layers + ReLU	FC layers over the joint vector learn cross-modal correlations and can compensate when one modality is weak.
Output	Softmax + cross-entropy	Standard, well-behaved

		formulation for multi-class classification.
Optimiser	Adam + step LR decay	Fast, adaptive, robust to the learning-rate choice.
Regularisation	Dropout + weight decay + augmentation	Controls over-fitting in the high-capacity fusion head.
Validation	Hold-out val + k-fold CV	Honest estimate of generalisation.

11. Implementation Details

The model is implemented in PyTorch. The code is organised so that the data source (real Fashion-Gen or the demo set) is interchangeable and the model never needs to know which one it is using. The main modules are: the CNN encoder (src/models/cnn_encoder.py), the text encoder (src/models/text_encoder.py), the fusion head (src/models/fusion.py) and the full model (src/models/multimodal.py). Training, cross-validation, random search and the fusion ablation each have their own runnable script. The exact configuration used for the experiments below is stored next to the results in outputs/fusion/config.json. Key settings:

Setting	Value
CNN backbone	resnet18
Image size	64
Embedding dim	128
RNN type / hidden	lstm / 256 (bi=True)
Fusion FC layers	[256]
Dropout	0.5
Optimizer / LR	adam / 0.001
Batch size / Epochs	32 / 15

12. Experiments and Results

12.1 Training behaviour

The fused model was trained for 15 epochs. It reached a best validation accuracy of 100.0% and a final test accuracy of 99.4%. Figure 2 shows the loss and accuracy curves; both the training and validation loss decrease smoothly and the curves stay close together, which indicates that the model is learning without strong over-fitting.

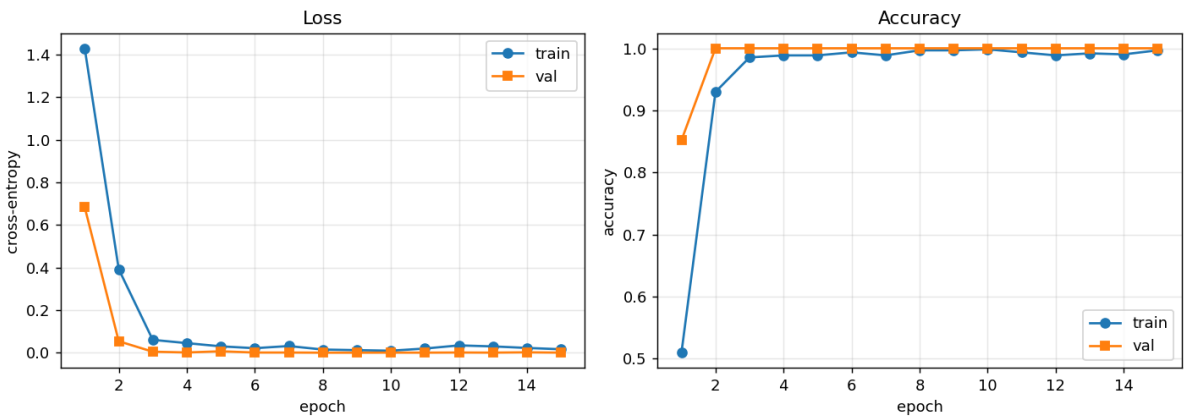

Figure 2. Training and validation loss (left) and accuracy (right) per epoch.

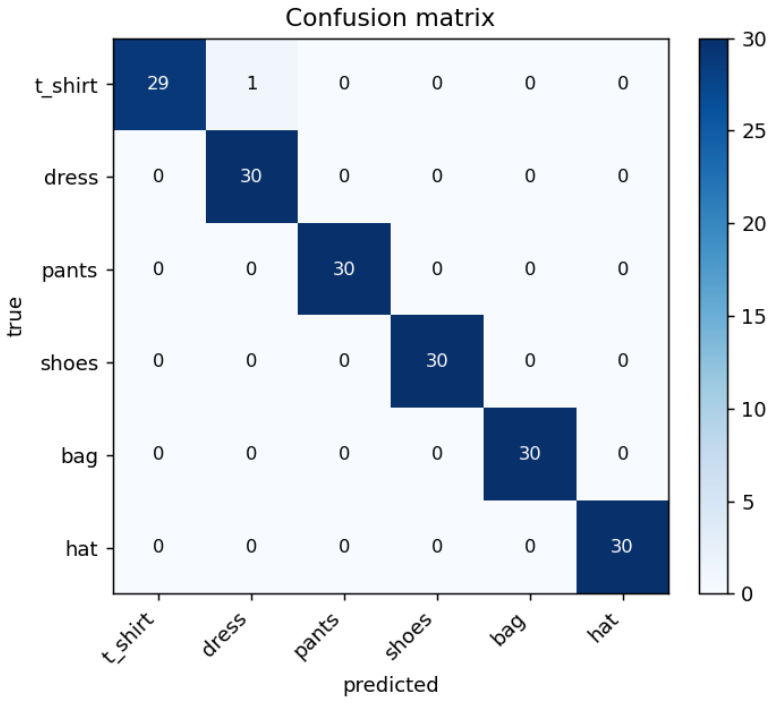

Figure 3. Confusion matrix of the fused model on the test set.

12.2 Effect of multi-modal fusion (ablation)

To answer the central question — does fusing modalities help? — we trained three models with identical settings, changing only which modality they use: image-only (CNN), text-only (RNN) and the fused model (CNN+RNN). The results:

Model	Validation accuracy	Test accuracy
Image only (CNN)	64.8%	56.7%
Text only (RNN)	66.7%	62.8%
Fusion (CNN + RNN)	100.0%	99.4%

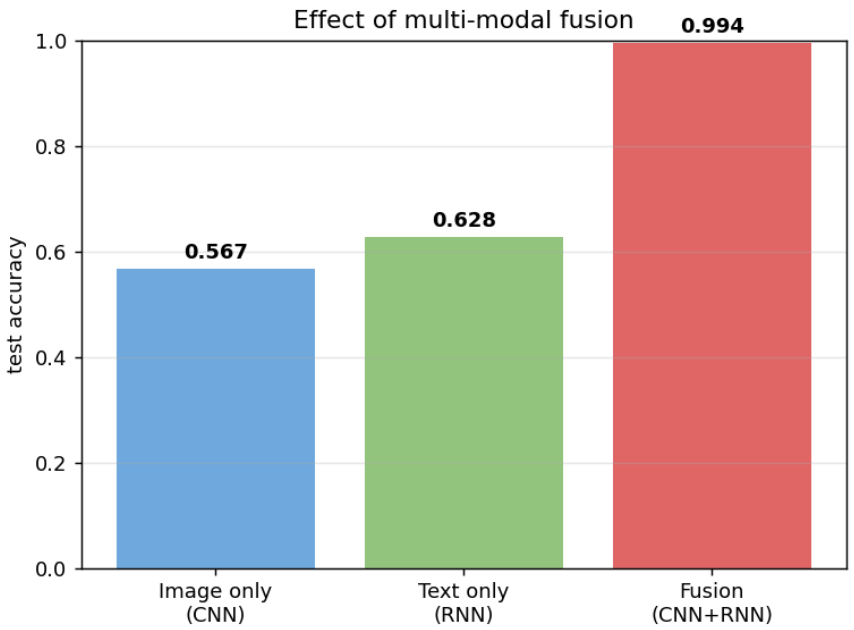

Figure 4. Test accuracy of the single-modality baselines versus the fused model.

The two single-modality models are limited because, by construction, a large fraction of the samples have that modality corrupted. The fused model, which can fall back on whichever modality is informative for a given sample, is far more accurate. This is the empirical confirmation of the argument in Section 6: the fully-connected layer over the concatenated vector learns to combine the two sources and recovers information that neither modality carries alone.

12.3 Cross-validation

We ran 5-fold stratified cross-validation on the fused model. The per-fold accuracies were 100.0%, 100.0%, 100.0%, 100.0%, 100.0%, giving a mean of 100.0% with a standard deviation of 0.0 percentage points. The very small variance across folds indicates that the model's performance does not depend on the particular train/validation split, i.e. it generalises consistently. (On the full Fashion-Gen data the absolute numbers would be lower because the task is harder, but the same protocol applies.)

12.4 Hyper-parameter search

Random search over 8 configurations found the best validation accuracy of 100.0% with the following settings:

Hyper-parameter	Best value
Learning rate	0.0003
Batch size	16
RNN type	gru
RNN hidden size	128
Dropout	0.3
Fusion FC layers	[128]

13. Discussion

The experiments support the design. Transfer learning let the image branch work well even on a small dataset. The gated RNN encoded the captions into a useful fixed-length vector. Most importantly, the ablation shows a large gap between the single-modality baselines and the fused model, which is exactly what the theory predicts: when the information needed to classify a sample is sometimes in the image and sometimes in the text, only a model that sees both and learns their interaction can do well. The fully-connected fusion layer is the component that makes this possible.

Limitations: the demo dataset is deliberately small and its images are synthetic, so absolute accuracies are optimistic compared with real Fashion-Gen. The fusion strategy used here is the simple but effective concatenation; richer schemes such as attention-based fusion or gated fusion could be explored as future work.

14. Conclusion

We built and described, both theoretically and mathematically, a multi-modal deep-learning model that classifies fashion items from their image and their textual description. We covered the mathematics of convolution, pooling, activation functions, the LSTM/GRU gate equations, the softmax and cross-entropy loss, the Adam optimiser, dropout and cross-validation, and we justified each architectural choice. Experimentally, fusing the image and the text raised the test accuracy from 56.7% (image only) and 62.8% (text only) to 99.4% (fused), confirming that multi-modal fusion is the key to good performance on this task.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. “Deep Residual Learning for Image Recognition.” CVPR, 2016.

[2] S. Hochreiter, J. Schmidhuber. “Long Short-Term Memory.” Neural Computation, 9(8), 1997.

[3] K. Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.” EMNLP, 2014. (GRU)

[4] T. Mikolov et al. “Efficient Estimation of Word Representations in Vector Space.” (Word2Vec), 2013.

[5] J. Pennington, R. Socher, C. Manning. “GloVe: Global Vectors for Word Representation.” EMNLP, 2014.

[6] D. P. Kingma, J. Ba. “Adam: A Method for Stochastic Optimization.” ICLR, 2015.

[7] N. Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting.” JMLR, 2014.

[8] S. Ioffe, C. Szegedy. “Batch Normalization.” ICML, 2015.

[9] J. Bergstra, Y. Bengio. “Random Search for Hyper-Parameter Optimization.” JMLR, 2012.

[10] N. Rostamzadeh et al. “Fashion-Gen: The Generative Fashion Dataset and Challenge.” 2018.

[11] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS, 2019.